
**CSE3023 WEB APPLICATION DEVELOPMENT**

**SEMESTER 2 SESSION 2025/2026**

**BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS**

# LAB REPORT 5

**PREPARED FOR :** DR ARIZAL

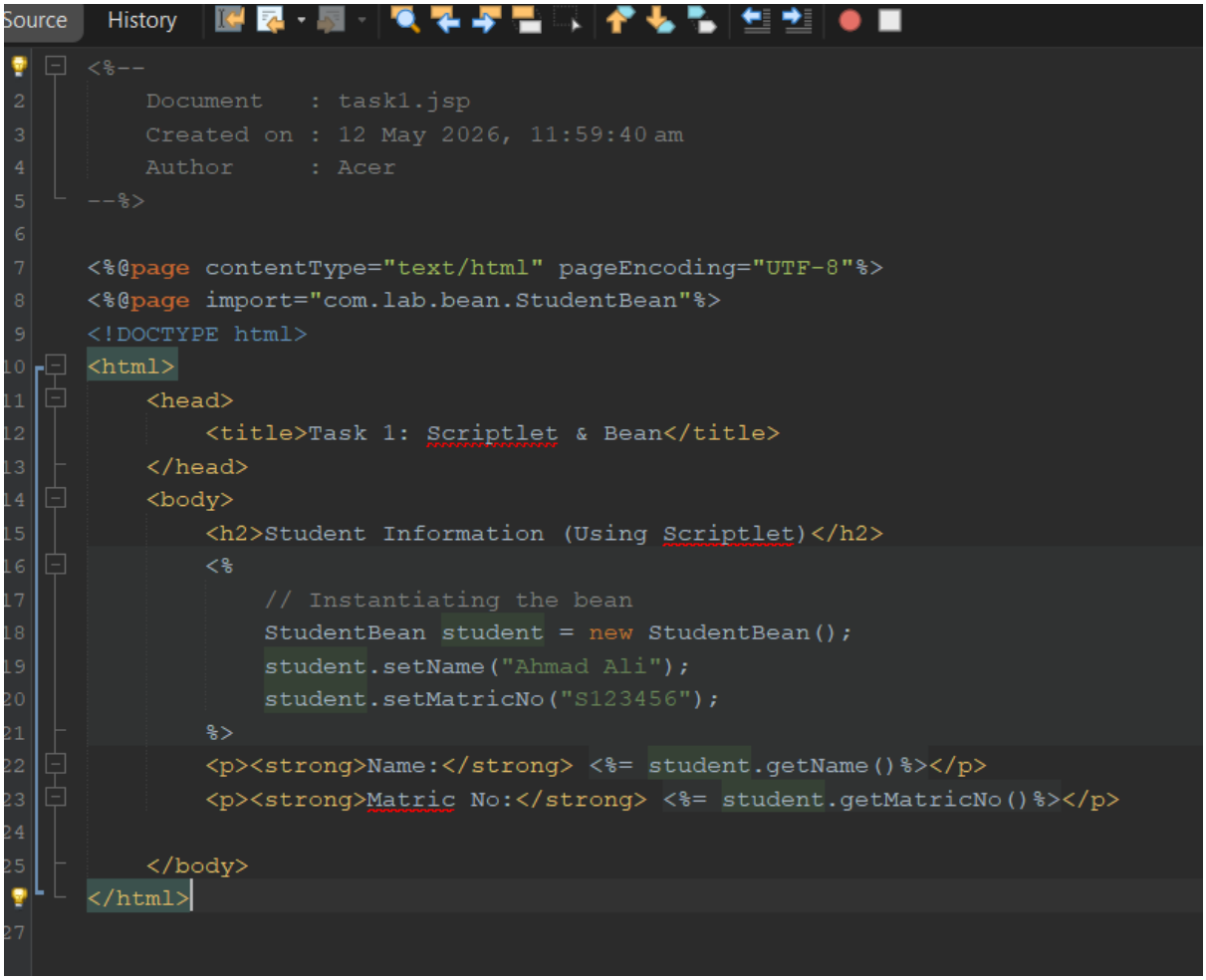
**PREPARED BY :** NOOR IZZA HAZIRA BINTI NORDDIN (S76025)

**DATE OF SUBMISSION** : 12 MEI 2026

TASK 1

Source History

```java
package com.lab.bean;

/**
 * Name : Noor Izza Hazira bt Norddin
 * Matric no : S76025
 */
public class StudentBean {

    private String name;
    private String matricNo;

    // Empty Default Constructor (Compulsory for JavaBeans)

    public StudentBean() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getMatricNo() {
        return matricNo;
    }

    public void setMatricNo(String matricNo) {
        this.matricNo = matricNo;
    }
}

```



```jsp
<%--
    Document   : task1.jsp
    Created on : 12 May 2026, 11:59:40 am
    Author     : Acer
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="com.lab.bean.StudentBean"%>
<!DOCTYPE html>
<html>
    <head>
        <title>Task 1: Scriptlet & Bean</title>
    </head>
    <body>
        <h2>Student Information (Using Scriptlet)</h2>
        <%
            // Instantiating the bean
            StudentBean student = new StudentBean();
            student.setName("Ahmad Ali");
            student.setMatricNo("S123456");
        %>
        <p><strong>Name:</strong> <%= student.getName()%></p>
        <p><strong>Matric No:</strong> <%= student.getMatricNo()%></p>

    </body>
</html>
```

Reflections

1. What is the main disadvantage of mixing Java scriptlets directly inside HTML codes?

   The main disadvantage of mixing Java scriptlets inside HTML is that it will results in "Spaghetti Code" because HTML and Java logic are mixed, so the page is difficult to read, maintain, and troubleshoot.

2. Why is it compulsory for a JavaBean class to have an empty default constructor?

   It is compulsory for a JavaBean an empty default constructor because the JSP container and other tools use "Introspection" and "Reflection" to manage the bean. The server must be able to automatically instantiate the class when we use the tag. The server depends on the presence of a default constructor to generate the object first because it does not know the specific values to send to a parameterized constructor. The server can use setter methods to set up the bean's properties after the object has been constructed. The JSP engine would not be able to build the bean instance without this empty constructor, which would cause a runtime error.

TASK 2

Source History

```java
package com.lab.bean;

/**
 * Name : Noor Izza Hazira bt Norddin
 * Matric no : S76025
 */
public class CalculatorBean {

    private double number1;
    private double number2;

    public CalculatorBean() {
    }

    public double getNumber1() {
        return number1;
    }

    public void setNumber1(double number1) {
        this.number1 = number1;
    }

    public double getNumber2() {
        return number2;
    }

    public void setNumber2(double number2) {
        this.number2 = number2;
    }
    // Custom method for logic

    public double getSum() {
        return number1 + number2;
    }
}
```

```html
<!DOCTYPE html>
<html>
    <body>
        <h2>Simple Calculator</h2>
        <form action="process.jsp" method="POST">
            Number 1: <input type="text" name="number1"><br><br>
            Number 2: <input type="text" name="number2"><br><br>
            <input type="submit" value="Calculate">
        </form>
    </body>
</html>
```

```jsp
<%--
    Document   : process
    Created on : 12 May 2026, 12:09:12 pm
    Author     : Acer
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <body>
        <h2>Calculation Result</h2>
        <jsp:useBean id="calc" class="com.lab.bean.CalculatorBean" />
        <jsp:setProperty name="calc" property="*" />
        <p>The sum of
            <jsp:getProperty name="calc" property="number1" /> and
            <jsp:getProperty name="calc" property="number2" /> is:
            <strong><jsp:getProperty name="calc" property="sum" /></strong>
        </p>
        <br><a href="calculator.html">Calculate Again</a>
    </body>
</html>
```

Reflections

1. Explain how the property="*" attribute works in the tag.

   The property="*" used for automatic data binding. The names of the attributes in the JavaBean and the input field names in an HTML form are automatically matched by the JSP container. The container automatically calls the matching setter method to set up the bean with user input if the name of a form field matches the name of a bean property. As a result, it is no longer necessary to write separate setProperty tags for each form field.

2. How do JSP action tags improve code readability compared to the scriptlets used in Task 1?

JSP action tags improve code readability by replacing complex Java scriptlets with an XML-like syntax that interacts naturally with HTML structure. This method removes "spaghetti code" by giving developers and web designers a clearer visual structure that makes it easier to differentiate between between display and logic. The code creates a better separation of concerns and minimises common syntax problems that frequently occur within scriptlets, such as missing semicolons or mismatched braces, by using tags like instead of raw Java blocks.

TASK 3

```
<%--
    Document   : jstl_test
    Created on : 12 May 2026, 12:19:24 pm
    Author     : Acer
--%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
<!DOCTYPE html>
<html>
    <head>
        <title>JSTL Date & Time Formatting</title>
    </head>
    <body>
        <h2>Testing JSTL Date & Time Formatting</h2>
        <jsp:useBean id="now" class="java.util.Date" />
        <p>1. Date Only (Default):
            <strong>
                <fmt:formatDate value="${now}" type="date" />
            </strong>
        </p>
        <p>2. Time Only (Default):
            <strong>
                <fmt:formatDate value="${now}" type="time" />
            </strong>
        </p>
        <p>3. Both Date & Time:
            <strong>
                <fmt:formatDate value="${now}" type="both" />
            </strong>
        </p>
        <p>4. Long Date & Short Time:
            <strong>
                <fmt:formatDate value="${now}" type="both" dateStyle="long"
                                timeStyle="short" />
            </strong>
```

```
            </p>
            <p>5. Custom Pattern (dd MMM yyyy, hh:mm a):
                <strong>
                    <fmt:formatDate value="${now}" pattern="dd MMMM yyyy, hh:mm:ss a" />
                </strong>
            </p>
        </body>
    </html>
```

# Testing JSTL Date & Time Formatting

1. Date Only (Default): **May 12, 2026**
2. Time Only (Default): **12:23:32 PM**
3. Both Date & Time: **May 12, 2026, 12:23:32 PM**
4. Long Date & Short Time: **May 12, 2026, 12:23 PM**
5. Custom Pattern (dd MMM yyyy, hh:mm a): **12 May 2026, 12:23:32 PM**

Reflections

1. What is the purpose of the uri and prefix attributes in the taglib directive?

   The JSP container can locate the functional definitions of the tag library precisely because uri acts like a unique address. The prefix attribute functions as a shorthand or local nickname for that library.

2. Why do we need separate taglib directives for core and fmt?

   We need separate directives for core and fmt because JSTL is modularized into many functional parts to maintain the applications' efficiency and organisation. The fmt library focuses on to standardisation and the localised formatting of dates, numerals, and currencies, while the core library focuses on basic structural operations like looping, conditionals, and URL management. Each must be declared separately in order for the JSP page to access its unique features because they have various URIs and offer different sets of tags.

TASK 4

```jsp
<%--
    Document   : jstl_test
    Created on : 12 May 2026, 12:19:24 pm
    Author     : Acer
--%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
    <head>
        <title>JSTL Core Tags</title>
    </head>
    <body>
        <h2>JSTL Conditional Test</h2>
        <c:set var="score" value="85" />
        <p>Student Score: <c:out value="${score}" /> marks.</p>
        <c:if test="${score >= 50}">
            <p style="color: green;"><strong>Status: Pass! Congratulations.</strong>
            </p>
        </c:if>
        <c:if test="${score < 50}">
            <p style="color: red;"><strong>Status: Fail. Please try again.</strong>
            </p>
        </c:if>
    </body>
</html>
```

Reflections

1. How does the tag help in preventing Cross-Site Scripting (XSS) attacks?

   The tag prevents XSS attacks by automatically escaping XML/HTML values. Special characters like < and > are automatically converted into the appropriate HTML entities (such as < and >). This guarantees that any dangerous scripts that a user enters are handled by the browser as harmless text rather than being executed as active co

2. The tag does not have an ”else” statement. Which JSTL tags should be used if you need an if-else logic structure?

   <c:choose>: Contains the conditional structure.

   <c:when>: Indicates the condition to be tested, either "if" or "else if."

   <c:otherwise>: : Serves as the "else" block and only runs in the event that none of the previous <c:when> conditions are satisfied.

TASK 5

```java
package com.lab.controller;

import com.lab.bean.StudentBean;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;

/**
 * Name : Noor Izza Hazira bt Norddin
 * Matric no : S76025
 */
import javax.servlet.http.HttpServletResponse;

public class StudentListServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

// Create a list to hold student objects
        List<StudentBean> stdList = new ArrayList<>();

// Adding sample data
        StudentBean s1 = new StudentBean();
        s1.setName("Ali bin Abu");
        s1.setMatricNo("S111");
        StudentBean s2 = new StudentBean();
        s2.setName("Siti Aminah");
        s2.setMatricNo("S222");
        StudentBean s3 = new StudentBean();
        s3.setName("Muthusamy");
        s3.setMatricNo("S333");
        stdList.add(s1);
        stdList.add(s2);
```

```java
// Adding sample data
        StudentBean s1 = new StudentBean();
        s1.setName("Ali bin Abu");
        s1.setMatricNo("S111");
        StudentBean s2 = new StudentBean();
        s2.setName("Siti Aminah");
        s2.setMatricNo("S222");
        StudentBean s3 = new StudentBean();
        s3.setName("Muthusamy");
        s3.setMatricNo("S333");
        stdList.add(s1);
        stdList.add(s2);
        stdList.add(s3);
// Share data with JSP
        request.setAttribute("listData", stdList);
// Forward to View
        RequestDispatcher rd = request.getRequestDispatcher("displayList.jsp");
        rd.forward(request, response);
    }
}
```

```jsp
<%--
    Document   : displayList
    Created on : 12 May 2026, 12:30:27 pm
    Author     : Acer
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
    <head>
        <title>JSTL Loop Collection</title>
    </head>
    <body>
        <h2>Registered Students List</h2>
        <table border="1" cellpadding="8">
            <thead>
                <tr style="background-color: lightgray;">
                    <th>No.</th>
                    <th>Name</th>
                    <th>Matric Number</th>
                </tr>
            </thead>
            <tbody>
                <c:forEach items="${listData}" var="student" varStatus="status">
                    <tr>
                        <td>${status.count}</td>
                        <td>${student.name}</td>
                        <td>${student.matricNo}</td>
                    </tr>
                </c:forEach>
            </tbody>
        </table>
    </body>
</html>
```

# Registered Students List

| No. | Name | Matric Number |
|---|---|---|
| 1 | Ali bin Abu | S111 |
| 2 | Siti Aminah | S222 |
| 3 | Muthusamy | S333 |

Reflections

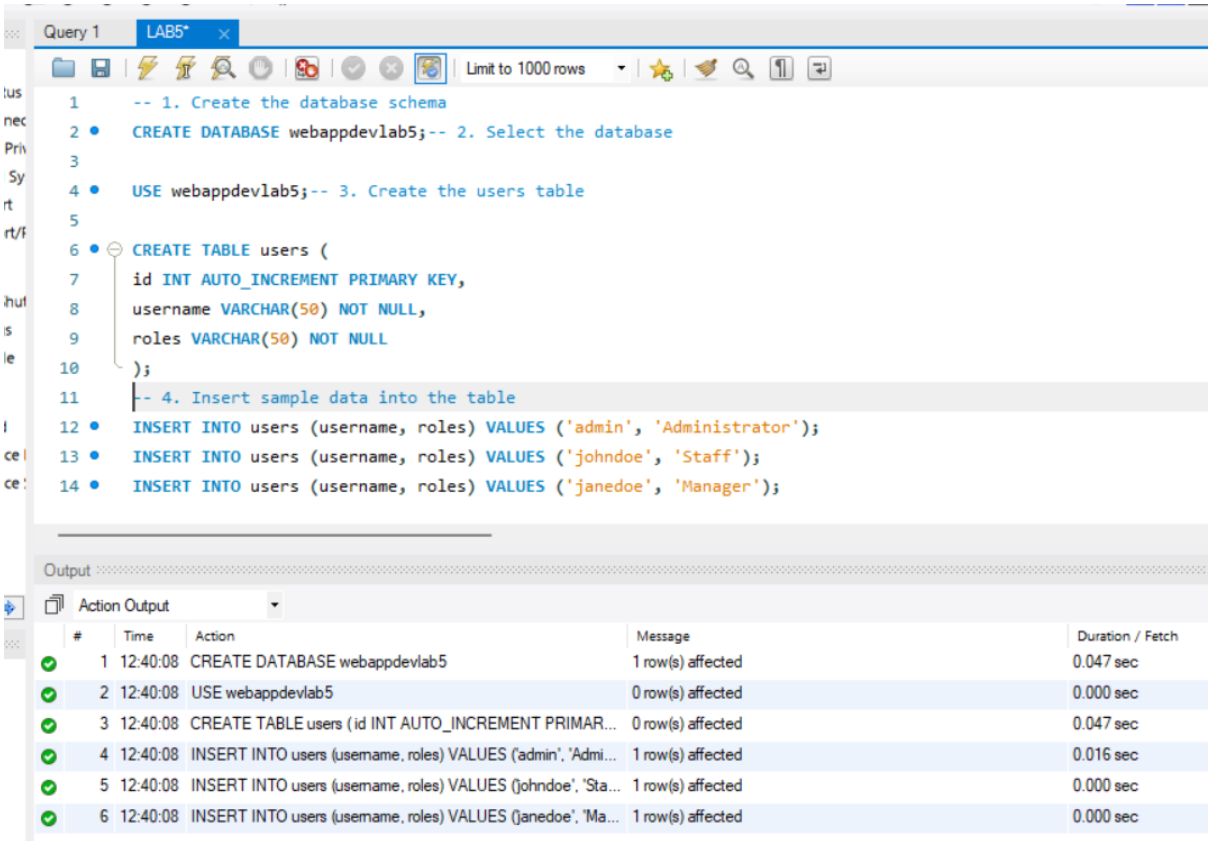
1. Describe how the items and var attributes work together in the tag.

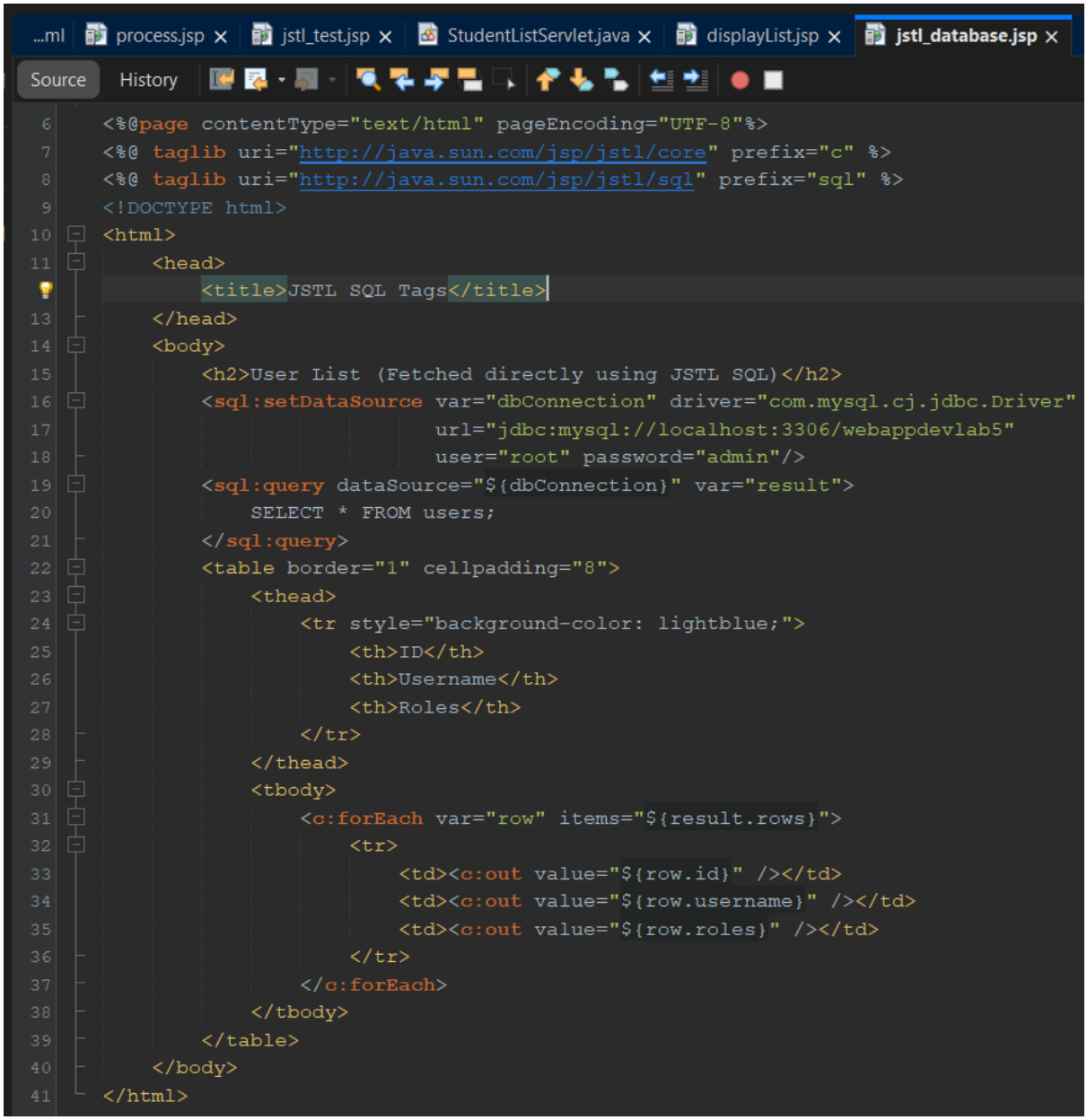
   The data list is provided by the items attribute, and during looping, each individual value is stored in var. One item is assigned to var for each iteration of the loop.

2. Why is it important to run the Servlet instead of the JSP file directly in this task?

   It is because the processing logic is handled first by Servlet. The controller layer is skipped while using JSP, which interrupts the MVC framework and could result in missing or unprocessed data.

TASK 6






```jsp
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
<!DOCTYPE html>
<html>
    <head>
        <title>JSTL SQL Tags</title>
    </head>
    <body>
        <h2>User List (Fetched directly using JSTL SQL)</h2>
        <sql:setDataSource var="dbConnection" driver="com.mysql.cj.jdbc.Driver"
                           url="jdbc:mysql://localhost:3306/webappdevlab5"
                           user="root" password="admin"/>
        <sql:query dataSource="${dbConnection}" var="result">
            SELECT * FROM users;
        </sql:query>
        <table border="1" cellpadding="8">
            <thead>
                <tr style="background-color: lightblue;">
                    <th>ID</th>
                    <th>Username</th>
                    <th>Roles</th>
                </tr>
            </thead>
            <tbody>
                <c:forEach var="row" items="${result.rows}">
                    <tr>
                        <td><c:out value="${row.id}" /></td>
                        <td><c:out value="${row.username}" /></td>
                        <td><c:out value="${row.roles}" /></td>
                    </tr>
                </c:forEach>
            </tbody>
        </table>
    </body>
</html>
```

# User List (Fetched directly using JSTL SQL)

| ID | Username | Roles |
|---|---|---|
| 1 | admin | Administrator |
| 2 | johndoe | Staff |
| 3 | janedoe | Manager |

Reflections

1. The JSTL tag allows you to retrieve data without writing Servlets or DAO classes. Why is this considered a bad practice (violating the MVC pattern) for large-scale enterprise applications?

   Using JSTL SQL tags for direct database access is considered bad practice since it breaks the Model-View-Controller (MVC) architecture by combining database functionality (Model) directly with the display layer (View).
   This method produces high coupling in large-scale enterprise programmes, which makes it much more difficult to maintain, debug, and reuse the code across other modules. Additionally, avoiding Servlets and Data Access Objects (DAO) prevents a clear separation of concerns, which is necessary for creating online applications that are professional, scalable, and well-organized.

EXERCISE

payroll_view.jsp

```jsp
<%--
    Document   : payroll_view
    Created on : 12 May 2026, 2:50:33 pm
    Author     : Acer
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<!DOCTYPE html>
<html>
    <head>
        <title>Employee Payroll Display</title>
    </head>
    <body>

        <h2>Employee Payroll List</h2>

        <table border="1" cellpadding="8">

            <thead>
                <tr style="background-color: lightgray;">
                    <th>ID</th>
                    <th>Name</th>
                    <th>Department</th>
                    <th>Basic Salary</th>
                    <th>Status</th>
                </tr>
            </thead>

            <tbody>

                <c:forEach var="emp" items="${employeeList}">
```

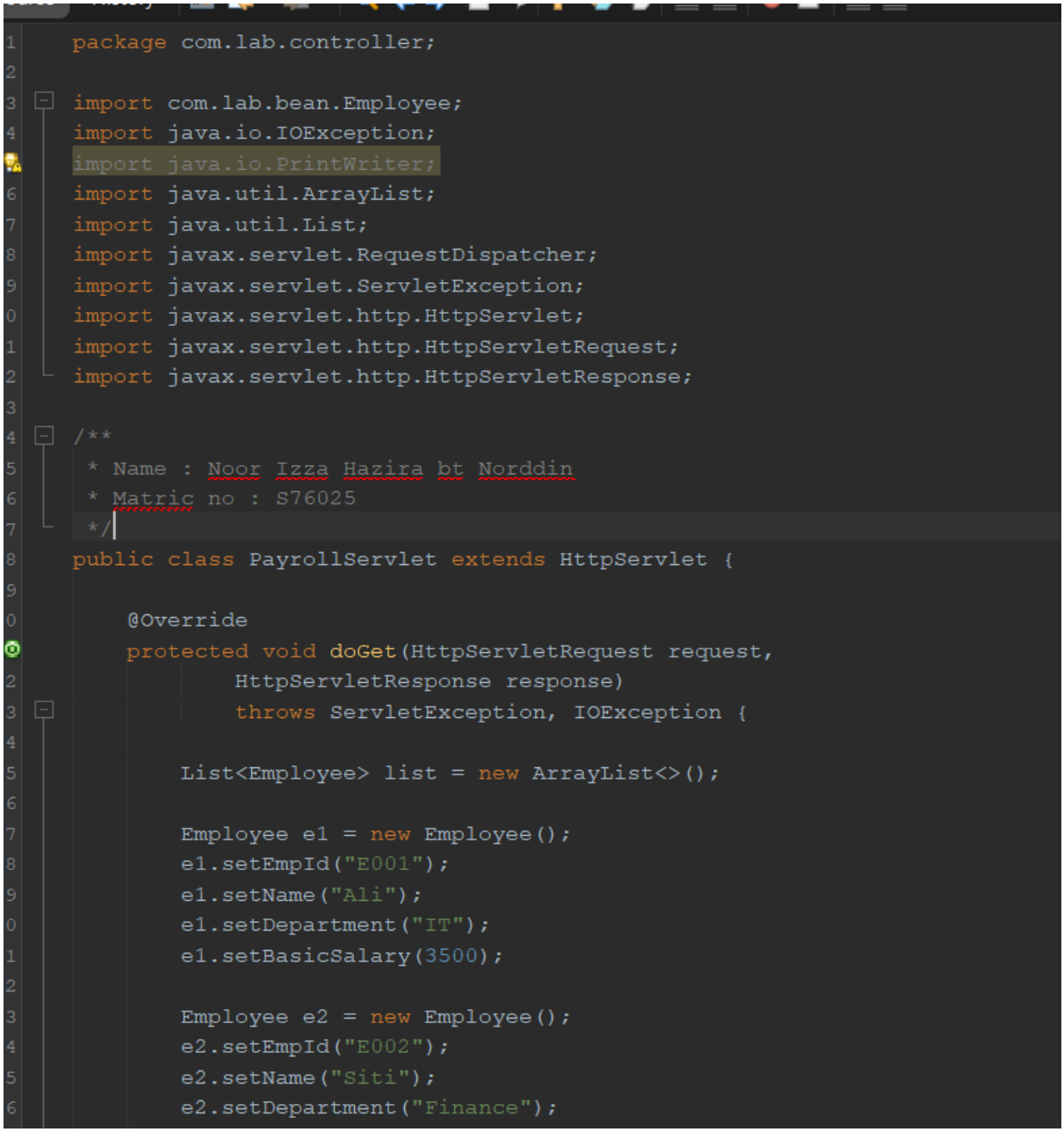
```
            <tbody>

                <c:forEach var="emp" items="${employeeList}">

                    <tr>
                        <td>${emp.empId}</td>
                        <td>${emp.name}</td>
                        <td>${emp.department}</td>
                        <td>RM ${emp.basicSalary}</td>

                        <td>
                            <c:choose>

                                <c:when test="${emp.basicSalary >= 3000}">
                                    Senior
                                </c:when>

                                <c:otherwise>
                                    Junior
                                </c:otherwise>

                            </c:choose>
                        </td>
                    </tr>

                </c:forEach>

            </tbody>

        </table>

    </body>
</html>
```

Employee.java

```java
package com.lab.bean;

/**
 * Name : Noor Izza Hazira bt Norddin
 * Matric no : S76025
 */
public class Employee {

    private String empId;
    private String name;
    private String department;
    private double basicSalary;

    public Employee() {
    }

    public String getEmpId() {
        return empId;
    }

    public void setEmpId(String empId) {
        this.empId = empId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDepartment() {
        return department;
    }
```

```java
    public void setDepartment(String department) {
        this.department = department;
    }

    public double getBasicSalary() {
        return basicSalary;
    }

    public void setBasicSalary(double basicSalary) {
        this.basicSalary = basicSalary;
    }
}
```

PayrollServlet.java



```java
package com.lab.controller;

import com.lab.bean.Employee;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * Name : Noor Izza Hazira bt Norddin
 * Matric no : S76025
 */
public class PayrollServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
            HttpServletResponse response)
            throws ServletException, IOException {

        List<Employee> list = new ArrayList<>();

        Employee e1 = new Employee();
        e1.setEmpId("E001");
        e1.setName("Ali");
        e1.setDepartment("IT");
        e1.setBasicSalary(3500);

        Employee e2 = new Employee();
        e2.setEmpId("E002");
        e2.setName("Siti");
        e2.setDepartment("Finance");
```

```
        Employee e2 = new Employee();
        e2.setEmpId("E002");
        e2.setName("Siti");
        e2.setDepartment("Finance");
        e2.setBasicSalary(2800);

        Employee e3 = new Employee();
        e3.setEmpId("E003");
        e3.setName("Kumar");
        e3.setDepartment("HR");
        e3.setBasicSalary(4000);

        Employee e4 = new Employee();
        e4.setEmpId("E004");
        e4.setName("John");
        e4.setDepartment("Marketing");
        e4.setBasicSalary(2500);

        Employee e5 = new Employee();
        e5.setEmpId("E005");
        e5.setName("Nurul");
        e5.setDepartment("Admin");
        e5.setBasicSalary(3200);

        list.add(e1);
        list.add(e2);
        list.add(e3);
        list.add(e4);
        list.add(e5);

        request.setAttribute("employeeList", list);

        RequestDispatcher rd = request.getRequestDispatcher("payroll_view.jsp");
        rd.forward(request, response);
    }
}
```

# Employee Payroll List

| ID | Name | Department | Basic Salary | Status |
|---|---|---|---|---|
| E001 | Ali | IT | RM 3500.0 | Senior |
| E002 | Siti | Finance | RM 2800.0 | Junior |
| E003 | Kumar | HR | RM 4000.0 | Senior |
| E004 | John | Marketing | RM 2500.0 | Junior |
| E005 | Nurul | Admin | RM 3200.0 | Senior |